

Meeting 6. Taught, Not Written

Act 1 The Slope Again · Act 2 The Update Rule · Act 3 The Learning Rate

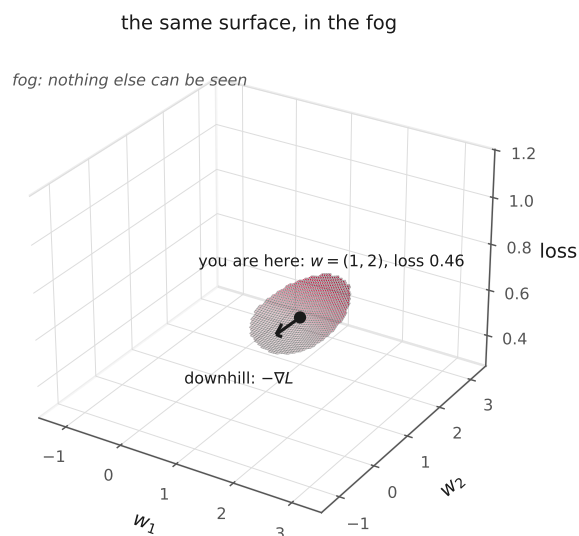
A unit's weights and bias determine its predictions. The labeled data let us measure how those predictions go wrong, and the gradient tells us how the loss responds to a small change in each parameter. We can use that information to change the parameters, make new predictions, and repeat. This procedure is **gradient descent**.

Act 1. The Slope Again

1.1 Finding a direction in the fog

Imagine standing on a hillside in thick fog. You cannot see the valley, but you can feel the slope under your feet. A small step downhill gives you a new place to stand. There you check the slope again: the direction that was downhill a few steps ago need not still be downhill here.

For the unit, a place on the hillside is a setting of its parameters, and height is the loss. The gradient supplies the local slopes. This lets the unit adjust its parameters without first drawing the whole loss surface or testing every setting on a grid.



This picture varies two weights while holding the bias fixed. The height is the loss, not a third parameter. Training the bias as well means moving in three parameter coordinates; drawing their loss would require a fourth axis.

The **gradient points uphill**, in the direction of steepest local increase in the loss, measured in the chosen parameter coordinates. We move against it to go downhill. Each partial derivative gives the slope along one coordinate. Together, these components form one gradient vector. Thinking of them as east–west and north–south components can help; the speed of the actual movement also depends on the learning rate.

1.2 What one row contributes

We use temperature $T = 1$. On row i , the unit computes

$$s_i = \mathbf{w} \cdot \mathbf{x}_i + b = \mathbf{w} \cdot \mathbf{x}_i - \theta, \quad \hat{y}_i = \sigma(s_i) = \frac{1}{1 + e^{-s_i}}.$$

The label y_i is either 0 or 1. We call

$$e_i = \hat{y}_i - y_i$$

the row's **error**. It is a signed difference between a probability and a label. It is not the row's loss, and it is not just a mark saying whether the 0-or-1 prediction was correct.

For example, an output of 0.82 with label 1 gives error -0.18 . The classification is correct, but increasing the probability would still improve the fit to that label. With label 0, an output of 0.62 gives error $+0.62$: the probability needs to come down.

For the mean binary cross-entropy loss from meeting 4,

$$L = \frac{1}{n} \sum_{i=1}^n [-y_i \log \hat{y}_i - (1 - y_i) \log(1 - \hat{y}_i)],$$

the derivative with respect to weight w_j is

$$\frac{\partial L}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n e_i x_{ij}.$$

Multiply each row's error by that row's j -th input, then average the products. Here n is the number of rows and x_{ij} is input j on row i .

Why multiply? The error tells us how the row's loss changes with its score. The input tells us how that score changes with the weight: changing w_j by a small amount h changes this row's score by hx_{ij} . If $x_{ij} = 0$, changing w_j cannot affect this row's score at all. Its contribution to this weight's derivative is zero, even if its prediction is badly wrong.

Getting a question wrong on a short quiz and getting it wrong on a heavily weighted exam can have different effects on a course grade. That is a useful way to remember the multiplication: the size of the error is only part of its effect. In the unit, the multiplier is the input value. A large numerical input does not by itself mean an important feature. Measuring the same length in centimeters instead of meters changes its numerical size without making it more important.

The bias contributes to every score with multiplier 1, so

$$\frac{\partial L}{\partial b} = \frac{1}{n} \sum_i e_i. \quad \frac{\partial L}{\partial \theta} = -\frac{1}{n} \sum_i e_i.$$

The threshold's minus sign comes from $s_i = \mathbf{w} \cdot \mathbf{x}_i - \theta$. The bias and threshold describe the same parameter, with $b = -\theta$; we train one or the other, not both independently.

1.3 Averaging conflicting feedback

Suppose course evaluations say both “too hard” and “too easy.” Changing the difficulty in response to one comment may move against another. A unit faces a similar problem: one row may call for a weight to increase while another calls for it to decrease. The average combines those requests, including their sizes.

Or consider a haircut. Half your friends love it; half think it is terrible. There is no obvious adjustment suggested by the head count alone. For the unit, each contribution has a numerical size, so equal numbers of positive and negative contributions need not cancel. Their magnitudes matter too.

A student could say, “Keep teaching the same way, but change your haircut.” Those comments concern different things you might change. Similarly, each weight gets its own average $\frac{1}{n} \sum_i e_i x_{ij}$. We do not average all the inputs together and make every weight react in the same way.

A small average can hide disagreement. Two rows have identical inputs but opposite labels. At output 0.5, their errors are +0.5 and −0.5. The errors cancel, and so do their contributions to every weight derivative. Both predictions remain uncertain. A small gradient does not mean that every row has a small error, or that every classification is correct.

Act 2. The Update Rule

2.1 Taking a step

Choose a positive **learning rate** η (the Greek letter eta). For each weight, subtract the learning rate times its derivative:

$$w_j^{\text{new}} = w_j^{\text{old}} - \eta \frac{\partial L}{\partial w_j}.$$

For the bias, use the same rule:

$$b^{\text{new}} = b^{\text{old}} - \eta \frac{\partial L}{\partial b}.$$

All derivatives are evaluated at the same old setting. Compute the gradient first, then update the parameters together. Recomputing a derivative halfway through the update would give a different procedure.

If a derivative is positive, increasing that parameter locally raises the loss, so the update decreases it. If the derivative is negative, the update increases the parameter. The minus sign handles both cases. At a setting with a nonzero gradient, a sufficiently small positive learning rate decreases the loss. A large step can overshoot the region where the local slope is a useful guide.

In threshold notation, the rule becomes

$$\theta^{\text{new}} = \theta^{\text{old}} + \eta \frac{1}{n} \sum_i e_i.$$

The plus sign is worth checking against the score. A positive mean error means that the mean predicted probability exceeds the mean label. Raising θ lowers every score, with the weights held fixed, and therefore lowers every predicted probability.

2.2 One complete update on AND

Example 2.1 From three correct rows to four

Start at $\mathbf{w} = (1, 2)$, $\theta = 1.5$. The unit predicts 1 when its score is at least zero. Its initial scores on the four AND rows are -1.5 , $+0.5$, -0.5 , and $+1.5$, so it gets $(0, 1)$ wrong. The loss is about 0.4627.

Using the probabilities rounded to two decimal places gives this calculation:

x_1	x_2	y	$\hat{y}$	$e = \hat{y} - y$	ex_1	ex_2
0	0	0	0.18	+0.18	0.00	0.00
0	1	0	0.62	+0.62	0.00	+0.62
1	0	0	0.38	+0.38	+0.38	0.00
1	1	1	0.82	-0.18	-0.18	-0.18
mean				+0.25	+0.05	+0.11

Every mean divides by four, including the zero entries. The gradient with respect to (w_1, w_2, θ) is approximately $(0.05, 0.11, -0.25)$.

At learning rate $\eta = 2$, the update is

$$\begin{aligned}w_1^{\text{new}} &= 1 - 2(0.05) = 0.90, \\w_2^{\text{new}} &= 2 - 2(0.11) = 1.78, \\\theta^{\text{new}} &= 1.5 - 2(-0.25) = 2.00.\end{aligned}$$

The updated score is $s = 0.90x_1 + 1.78x_2 - 2.00$:

x_1	x_2	label	new score	prediction
0	0	0	-2.00	0
0	1	0	-0.22	0
1	0	0	-1.10	0
1	1	1	+0.68	1

All four classifications are now correct. The loss has fallen to about 0.3533. Using unrounded probabilities changes the new weights slightly, to approximately $(0.90244, 1.77998)$, and gives the same loss to four decimal places.

The threshold changed the most: it rose by 0.50, while the weights fell by 0.10 and 0.22. Its direction comes from the positive mean error. This is a statement about the probabilities: three of the four classifications were already correct before the update.

One step need not fix every classification. From this same starting setting, $\eta = 0.5$ still leaves $(0, 1)$ on the wrong side of the line. The update aims to reduce the mean loss; it makes no promise to correct a particular row in one step.

2.3 Putting the update in a loop

After changing the parameters, recompute the predictions and the gradient. The repeated calculation is:

1. Compute each row's probability $\hat{y}_i$ at the current setting.
2. Compute each error $e_i = \hat{y}_i - y_i$.
3. For each weight, average $e_i x_{ij}$; for the bias, average e_i .
4. Subtract η times each derivative from its parameter.
5. Repeat until the chosen stopping condition is met.

This is **full-batch gradient descent**: every update uses every training row. It is like completing the whole practice quiz before using its feedback to adjust how you study.

With NumPy arrays, the loop can be written as follows. Here **X** has one row per example and one column per input; **y** is a one-dimensional array of labels. `unit.probability(X)` returns the corresponding one-dimensional array of probabilities at $T = 1$, and `unit.w` holds the weights.

```
import numpy as np

unit.w = np.asarray(unit.w, dtype=float)
for step in range(n_steps):
    p = unit.probability(X)
    err = p - y
    grad_w = (X.T @ err) / len(y)
    grad_theta = -err.mean()
    unit.w -= eta * grad_w
    unit.theta -= eta * grad_theta
```

For arrays, `p - y` subtracts corresponding entries. If `p` contains `[0.18, 0.62, 0.38, 0.82]` and `y` contains `[0, 0, 0, 1]`, the result contains `[0.18, 0.62, 0.38, -0.18]`. Plain Python lists do not support this subtraction.

The expression `X.T @ err` computes a dot product for each input column. Its j -th entry is $\sum_i x_{ij} e_i$. Dividing by `len(y)` gives the required average. It is the *average of the products*, not the product of the average input and the average error. NumPy performs the row arithmetic inside its array operations; writing one line does not remove that work.

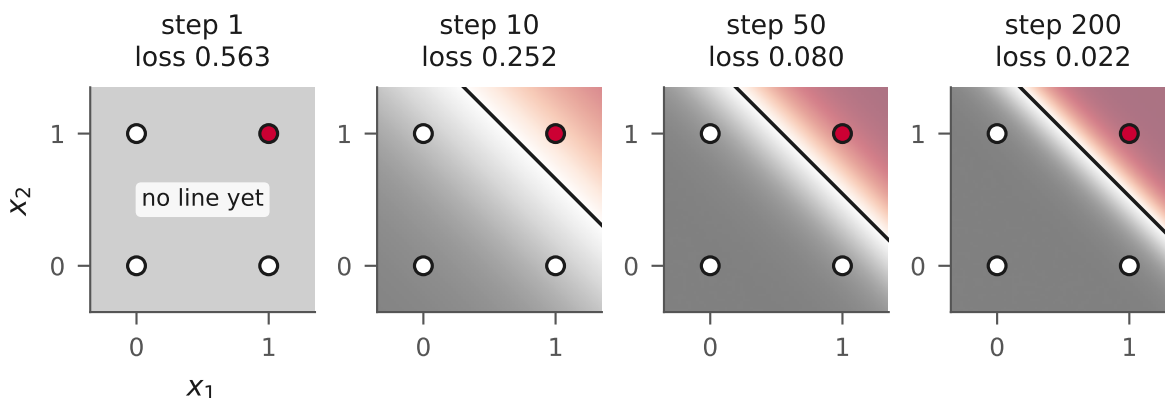
Both gradients are computed before either parameter update, so they use the same predictions. The last line could equivalently be written `unit.theta += eta * err.mean()`.

If there are 100 practice questions, you might instead review your mistakes after each group of ten. Training can also update from a subset of rows: **mini-batch gradient descent** uses the mean gradient contribution from a small batch. A randomly sampled batch gives an estimate of the full gradient. It costs less per update, but its direction varies with the sample and an individual update can increase the full-dataset loss.

2.4 Watching the unit learn a gate

Start a fresh AND unit with all parameters zero and use $\eta = 4$. Every initial probability is 0.5. The errors are $+0.5, +0.5, +0.5, -0.5$. For either weight, the positive and negative contributions cancel; the mean error is 0.25. The first step therefore leaves both weights at zero and raises the threshold to 1.

At this point every score is -1 . The unit predicts 0 on all four rows, which matches the three zero labels. It has no decision line yet: with both weights zero, changing the inputs cannot change its prediction. Later steps give the weights nonzero values and allow the unit to distinguish the corners.



The line separates predictions 0 and 1; the shading shows the probabilities. AND treats its two inputs symmetrically. Starting with equal weights and applying the same rule to each keeps them equal.

updates	loss	(w_1, w_2, θ)
1	0.5633	(0.000, 0.000, 1.000)
10	0.2523	(1.812, 1.812, 3.002)
50	0.0801	(4.277, 4.277, 6.607)
200	0.0216	(6.976, 6.976, 10.637)
2,000	0.0021	(11.628, 11.628, 17.610)

The unit first classifies all four rows correctly after four updates. The later updates keep reducing the loss by making the correct predictions more confident. The boundary can change very little while the probabilities continue to change substantially.

The same loop can train separate units on OR and NAND. Keep the inputs, zero initialization, and learning rate 4; change the labels:

table	first all-correct step	loss at step 200	all rows correct?
AND	4	0.0216	yes
OR	5	0.0116	yes
NAND	4	0.0216	yes

Each run learns its own parameters. The labels specify which function the run should fit; we do not supply a separate hand-chosen update rule for each gate.

2.5 When to stop

For AND, a finite sigmoid score always gives a probability strictly between 0 and 1. Once every row has a score strictly on the correct side of zero, scaling up all three parameters makes every probability closer to its label. The loss approaches zero but never reaches zero at finite parameter values. With all three parameters free, this problem has no finite loss-minimizing setting.

Waiting for the gradient to become exactly zero would therefore give no mathematical stopping point for this AND run. We can instead choose a number of updates, a tolerance for the gradient's size, or a tolerance for the improvement in loss. The condition should match what we need: reproducing a truth table and fitting its probabilities to high confidence are different requirements.

On other datasets, a zero gradient can occur at finite parameters, including when individual errors cancel. Also, tiny parameter changes may simply mean that the learning rate is tiny. Neither observation by itself proves that every prediction is good.

2.6 Cost and the weather example

Computing all components of the gradient takes a pass over the data. For n rows with d inputs, the direct calculation uses work proportional to nd , as does computing all predictions and the loss. The gradient does not require a separate dataset pass for each parameter. The total training cost also depends on how many updates are needed.

On AND, 50 gradient updates from zero at $\eta = 4$ reach loss 0.080. For comparison, the meeting-5 grid tests $25^3 = 15,625$ settings and finds loss 0.096; the displayed random-search run reaches about 0.11 after 1,000 guesses. Counting data passes gives an approximate cost comparison. These are results for the specified runs: the grid and random search use a bounded search box, while this gradient run is not restricted to that box.

For the Newark weather data, the inputs are daily high and low temperatures, and the label says whether the date falls in the warm half of the year, May through October. We use the same 6,865 complete rows as before; the date itself is not an input. Start from the hand line high + low = 120, expressed at $T = 1$ as $\mathbf{w} = (0.1, 0.1)$ and $b = -12$. Updating all three parameters at $\eta = 0.002$ gives:

updates	loss	days classified correctly
0	0.2447	89.7%
100	0.2234	90.6%
500	0.2181	90.8%
40,000	0.2174	91.0%

Much of the measured improvement arrives in the first few hundred updates. Here the loss settles near 0.22, well above zero. The dataset contains identical high-low pairs with opposite labels, so these two inputs cannot determine every label correctly, regardless of how long we train. The reported percentages are measured on the training rows; they do not measure performance on unseen days.

Act 3. The Learning Rate

3.1 How far does a step go?

The gradient gives a direction and a magnitude. The learning rate scales the change. If $\mathbf{q}$ contains all the parameters being trained, then

$$\Delta \mathbf{q} = -\eta \nabla L, \quad \|\Delta \mathbf{q}\| = \eta \|\nabla L\|.$$

Here ∇L includes the derivatives for every component of $\mathbf{q}$, including the bias or threshold if it is free. A learning rate of 2 does not mean moving a distance of 2. At a fixed rate, the steps shrink when the gradient shrinks.

The slope is local. In the fog analogy, feeling downhill under one foot does not tell you how the ground behaves fifty meters away. For a small step, the first-order estimate is

$$L(\mathbf{q} - \eta \nabla L) \approx L(\mathbf{q}) - \eta \|\nabla L\|^2.$$

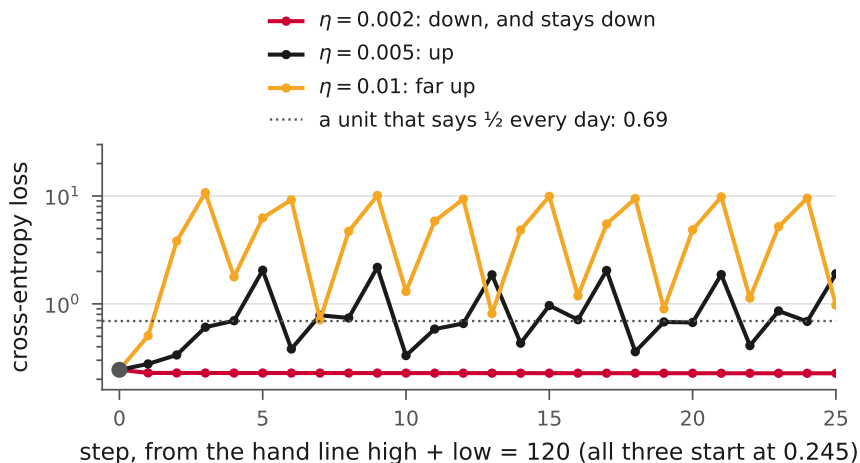
If the gradient is nonzero, this predicts a decrease. The approximation becomes less reliable as the step grows.

A very small learning rate can make progress slow. On AND, starting at $(1, 2, 1.5)$ with $\eta = 0.01$, the loss goes from 0.4627 to 0.4482 after 20 steps and 0.4037 after 100. The unit still misclassifies $(0, 1)$ at step 100.

3.2 Overshooting

An excessive rate can send the unit across a valley and up the other side. The next gradient may point back, producing repeated large swings. A direction that is downhill locally can still lead to a higher loss if followed too far.

The weather example makes this visible. From the same hand-line starting setting, $\eta = 0.002$ gives a steady decrease in the plotted run. At $\eta = 0.005$ the loss oscillates, and at $\eta = 0.01$ the swings are much larger.



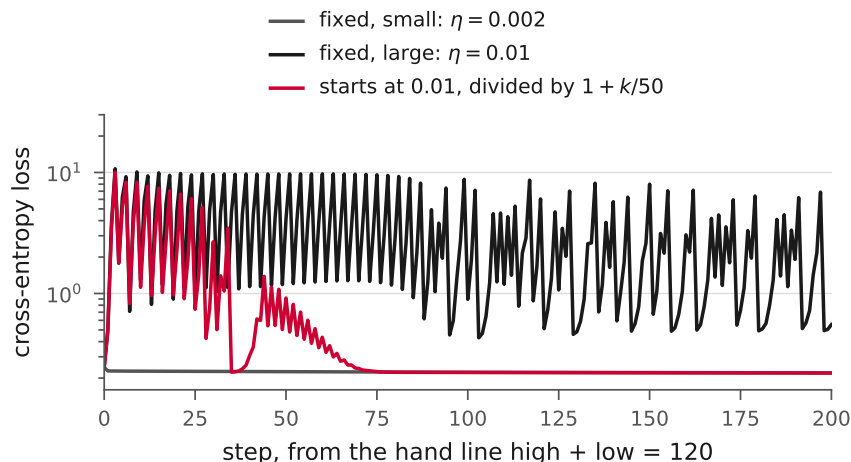
For $\eta = 0.01$, the first two updates take the loss from about 0.245 to 0.505 and then 3.821. The gradient formula has not changed; the step is too large for this setting and scale of inputs. A rate that works on the 0-or-1 inputs of AND need not work on temperatures measured in degrees.

3.3 Reducing the rate during training

We often make larger adjustments while exploring a problem, then smaller adjustments while refining an answer. A **learning-rate schedule** implements a specified change in the rate over time. One schedule used on the weather example is

$$\eta_k = \frac{0.01}{1 + k/50}, \quad k = 1, 2, 3, \dots$$

where k counts updates. The first rate is about 0.0098; at update 50 it is 0.005, and at update 200 it is 0.002.



The scheduled run initially overshoots too, but the oscillations subside as the rate decreases. After 200 updates its loss is about 0.2201, compared with 0.2207 for the fixed rate 0.002. The fixed small rate already works well here; the schedule shows how reducing a rate can stabilize a run that begins with overly large steps. A decreasing schedule does not automatically outperform a well-chosen constant rate.

Trying the interactive landscape. Reset to the same starting point when comparing rates. Take several steps, watch the loss, and check how the direction changes after each step. A single successful step is not enough to judge a rate.

The two-dimensional interactive fixes $\theta = 1.5$ for AND and $b = -12$ for weather. It updates only the two weights. The hand calculation and training runs above update all three parameters, so their paths and final losses need not match the interactive.

3.4 Learning analogies

The phrase “the neuron reacts to its mistakes” describes a computation: form errors, average their gradient contributions, and update the parameters. It is a useful way to talk about the loop. It does not establish that biological neurons or people learn by this exact rule.

Other intuitions from studying can suggest changes to training. Keeping some of the previous direction resembles **momentum**, which carries information from earlier updates into the next one. Working from easier examples toward harder ones suggests **curriculum learning**. To tell whether such a change helps, we need a specified algorithm and a comparison of its results.

For curriculum learning: Bengio et al., *Curriculum Learning* (2009).

In a larger network, **dropout** randomly omits units during training, temporarily removing their contributions and associated connections. The network is thus trained with different subsets of units present. Dropout does not mean discarding difficult rows or removing outliers from the data.

For the method and its motivation: Srivastava et al., *Dropout: A Simple Way to Prevent Neural Networks from Overfitting* (2014).

Attention gives another comparison. A teacher trying to follow every conversation in a room has to distribute attention somehow. In a Transformer’s attention mechanism, each position in a sequence can assign different numerical weights to information from the available positions. Within one such calculation, the weights are nonnegative and add to one, and the output is a weighted sum. The model forms many of these combinations, each with its own weights. The classroom comparison helps us picture distributing emphasis; the model’s “attention” is this numerical operation.

For the attention calculation: Vaswani et al., *Attention Is All You Need* (2017), Section 3.2.